Лекция 7. Задача о назначениях, венгерский алгоритм.

1. Введение

Задача о назначениях (assignment problem) – классическая комбинаторная оптимизационная задача. Представьте, что у нас есть n работников и n работ. Каждый работник может выполнить любую работу, но затраты (стоимость, время и т.п.) зависят от пары (работник, работа). Задана матрица C размера $n \times n$, где элемент $C[i][j]$ – стоимость выполнения работы j работником i. Требуется назначить каждого работника ровно на одну работу (и каждую работу – ровно одному работнику) так, чтобы суммарная стоимость всех назначений была **минимальна**.

Формально: найти перестановку π множества $\{1, \dots, n\}$, минимизирующую сумму

$$\sum_{i=1}^{n} C[i][\pi(i)].$$

Задача о назначениях является частным случаем транспортной задачи и задачи о максимальном потоке минимальной стоимости. Однако исторически для её решения был разработан специальный метод – **венгерский алгоритм** (Kuhn, 1955, Munkres, 1957), который эффективно использует специфику задачи и не требует общих алгоритмов потоков.

2. Основные свойства и идеи

2.1 Приведение матрицы

Ключевое наблюдение: если из всех элементов некоторой строки вычесть одну и ту же константу, то оптимальное решение не изменится (изменится только значение целевой функции на эту константу, умноженную на n, но относительный порядок сумм останется тем же). То же верно для столбцов.

Это позволяет **привести** матрицу к виду, где в каждой строке и каждом столбце есть хотя бы один ноль, а все элементы неотрицательны. Достаточно:

- Из каждой строки вычесть её минимальный элемент.
- Из каждого получившегося столбца вычесть его минимальный элемент.

Полученная матрица содержит нули и неотрицательные элементы. Если теперь удастся найти полное паросочетание (назначение) только по нулевым элементам, то оно будет оптимальным (стоимость 0 в приведённой матрице соответствует минимальной исходной стоимости). Если полного паросочетания из нулей нет, нужно продолжить преобразования, создавая новые нули.

2.2 Связь с паросочетаниями в двудольном графе

Рассмотрим двудольный граф: левая доля – работники (строки), правая доля – работы (столбцы). Ребро (i, j) существует, если $C[i][j] = 0$ (после приведения). Полное паросочетание в этом графе соответствует назначению с нулевой суммарной стоимостью в приведённой матрице, а значит, оптимальному исходному.

Если полного паросочетания нет, то нужно модифицировать матрицу так, чтобы появились новые нули, не нарушая неотрицательности. При этом уже существующие нули могут пропадать, но важно, что после преобразования оптимальное решение остаётся тем же.

3. Венгерский алгоритм (минимизация)

Венгерский алгоритм итеративно строит максимальное паросочетание в графе нулей и, если оно не полное, корректирует матрицу с помощью специальной процедуры пересчёта потенциалов. Рассмотрим классический вариант, использующий понятия чередующегося дерева и множеств.

3.1 Обозначения и структуры данных

Пусть n – количество строк/столбцов.

Введём:

- $u[i]$ – потенциал для строки i (сколько вычли из строки суммарно).
- $v[j]$ – потенциал для столбца j (сколько вычли из столбца суммарно).

Тогда текущее значение элемента $C'[i][j] = C[i][j] - u[i] - v[j]$ всегда неотрицательно.

Изначально $u[i] = \min_j C[i][j]$, затем после вычитания из строк вычитаем минимумы из столбцов: $v[j] = \min_i (C[i][j] - u[i])$.

В процессе алгоритма мы будем поддерживать текущее паросочетание $pairU[i]$ – для строки i номер столбца, с которым она спарена (или -1), и $pairV[j]$ – для столбца j номер строки (или -1).

Алгоритм работает по строкам (или по столбцам): пытаемся найти дополняющий путь для каждой свободной строки. Если не получается – модифицируем матрицу.

3.2 Поиск дополняющего пути (DFS)

Для текущей свободной строки s строим чередующееся дерево:

- Множество S (левая доля, достижимые вершины) инициализируем $\{s\}$.
- Множество T (правая доля, достижимые вершины) пусто.
- Используем массивы $prev$ для восстановления пути.

Процесс:

1. Ищем ребро (i, j) с $C'[i][j] = 0$ (нулевое в приведённой матрице), где $i \in S, j \notin T$.

2. Если нашли, добавляем j в T. Если j свободна ($pairV[j] = -1$), то найден дополняющий путь ($s \to \ldots \to j$). Инвертируем рёбра на пути (увеличиваем паросочетание).

3. Если j занята, пусть $i' = pairV[j]$. Добавляем i' в S (через ребро паросочетания) и продолжаем поиск.

Если на каком-то шаге не удаётся найти такое ребро, значит, дерево построено полностью, и паросочетание нельзя увеличить. В этом случае переходим к модификации матрицы.

3.3 Модификация матрицы (пересчёт потенциалов)

Пусть после построения дерева имеем:

- S – множество строк, достижимых из s.
- T – множество столбцов, достижимых из s (они все заняты вершинами из S?).

Рассмотрим все рёбра из S в столбцы, не входящие в T. Обозначим

$$delta = \min_{i \in S, j \notin T} C'[i][j].$$

Поскольку ни одно из этих рёбер не является нулевым (иначе бы мы его добавили в дерево), $delta > 0$.

Производим пересчёт:

- Для всех $i \in S$: $u[i] += delta$ (т.е. увеличиваем потенциал, что эквивалентно вычитанию delta из элементов строки).
- Для всех $j \in T$: $v[j] -= delta$ (т.е. уменьшаем потенциал, что эквивалентно добавлению delta к элементам столбца).

Посмотрим, как изменятся значения $C'[i][j] = C[i][j] - u[i] - v[j]$:

- Если $i \in S$, $j \in T$: изменение $= -delta + delta = 0$ (остаются без изменений).
- Если $i \in S$, $j \notin T$: изменение $= -delta - 0 = -delta$ (элементы уменьшаются на delta). Среди них хотя бы один (тот, на котором достигался минимум) становится нулевым.
- Если $i \notin S$, $j \in T$: изменение $= 0 + delta = +delta$ (элементы увеличиваются).
- Если $i \notin S, j \notin T$: изменение $= 0$ (остаются без изменений).

Важно: все C' остаются неотрицательными, так как мы вычитали $delta$ только из тех элементов, которые были не меньше delta. При этом новые нули появляются на границе S и не-T. Эти новые нули позволяют расширить дерево на следующей итерации.

3.4 Общая схема алгоритма

Повторяем для каждой строки i от 0 до $n-1$:

1. Попытаться найти дополняющий путь, начиная с i, используя текущие потенциалы u, v и текущее паросочетание. Если путь найден – увеличить паросочетание и перейти к следующей строке.

2. Если путь не найден – построить множества S и T, вычислить $delta$, обновить потенциалы и вернуться к шагу 1 (поиск пути для той же строки).

После обработки всех строк получаем полное паросочетание. Итоговые назначения хранятся в $pairU[i]$. Суммарная стоимость исходной задачи равна сумме $u[i] + v[j]$ по всем спаренным (i, j) (или просто сумме исходных стоимостей выбранных рёбер).

3.5 Псевдокод классического венгерского алгоритма (минимизация)

```
function hungarian(C):
   n = size(C)
   // Инициализация потенциалов
   u = array of n zeros
   v = array of n zeros
   for i = 0 to n-1:
      u[i] = min(C[i][j] for j)
      for j = 0 to n-1:
         C[i][j] -= u[i]
   for j = 0 to n-1:
      v[j] = min(C[i][j] for i)
      for i = 0 to n-1:
         C[i][j] -= v[j]

   pairU = array of n filled with -1
   pairV = array of n filled with -1

   for start = 0 to n-1:
      // строим чередующееся дерево для строки start
      S = boolean array of n filled false
      T = boolean array of n filled false
      prev = array of n filled with -1   // для восстановления пути
      queue = empty
      S[start] = true
      queue.push(start)
      found = false

      while not found and queue not empty:
         i = queue.pop()
```

```
        for j = 0 to n-1:
            if not T[j] and C[i][j] == 0:
                T[j] = true
                if pairV[j] == -1:
                    // нашли дополняющий путь
                    found = true
                    // восстановить путь и увеличить паросочетание
                    update_matching(j, i, prev, pairU, pairV)
                    break
                else:
                    i2 = pairV[j]
                    if not S[i2]:
                        S[i2] = true
                        prev[i2] = i
                        queue.push(i2)
        if found: break

    if not found:
        // модификация матрицы
        delta = INF
        for i in 0..n-1 where S[i]:
            for j in 0..n-1 where not T[j]:
                delta = min(delta, C[i][j])
        for i in 0..n-1:
            if S[i]:
                u[i] += delta
                for j in 0..n-1:
                    C[i][j] -= delta
        for j in 0..n-1:
            if T[j]:
                v[j] -= delta
                for i in 0..n-1:
                    C[i][j] += delta
        // повторить попытку для той же start (с новыми C)
        start--  // вернуться на шаг назад в цикле (или использовать while)

// построить результат
result = array of n
for i = 0 to n-1:
    j = pairU[i]
    result[i] = j
return result, total_cost = sum(original_C[i][result[i]])
```

Функция update_matching восстанавливает путь по prev и инвертирует рёбра. Обычно prev хранит для каждой строки предыдущую строку в пути.

```
function update_matching(j, i, prev, pairU, pairV):
  while i != -1:
    next_j = pairU[i]
    pairU[i] = j
    pairV[j] = i
    i = prev[i]   // предыдущая строка
    j = next_j
```

4. Пример работы алгоритма

Рассмотрим матрицу 4×4:

	1	2	3	4
1	5	8	6	4
2	3	2	1	3
3	6	7	5	4
4	4	2	3	5

Шаг 1. Приведение по строкам:

Минимумы: стр1:4, стр2:1, стр3:4, стр4:2. После вычитания:

	1	2	3	4
1	1	4	2	0
2	2	1	0	2
3	2	3	1	0
4	2	0	1	3

Шаг 2. Приведение по столбцам:

Минимумы: ст1:1, ст2:0, ст3:0, ст4:0. После вычитания:

	1	2	3	4
1	0	4	2	0
2	1	1	0	2
3	1	3	1	0
4	1	0	1	3

Теперь ищем полное паросочетание по нулям.

Итерация для строки 1:

Строим дерево из строки 1: $S = \{1\}$. Нули: $(1,1)$ и $(1,4)$. Берём $(1,1)$ →столбец 1 свободен? Да ($pairV[1] = -1$). Нашли дополняющий путь. Паросочетание: $\{(1,1)\}$.

Итерация для строки 2:

Начинаем с 2. Ищем нули: $(2,3)$. Столбец 3 свободен→добавляем $(2,3)$. Паросочетание: $\{(1,1),(2,3)\}$.

Итерация для строки 3:

Начинаем с 3. Нули: $(3,4)$. Столбец 4 свободен→добавляем $(3,4)$. Паросочетание: $\{(1,1),(2,3),(3,4)\}$.

Итерация для строки 4:

Начинаем с 4. Нули: $(4,2)$. Столбец 2 свободен→добавляем $(4,2)$. Полное паросочетание: все строки покрыты. Оптимальное назначение: $1 \to 1$,

$2 \to 3$, $3 \to 4$, $4 \to 2$. Стоимость исходной: $5 + 1 + 4 + 2 = 12$ (проверим: $5 + 1 + 4 + 2 = 12$). Можно убедиться, что это минимум.

В этом примере алгоритм завершился сразу, без модификаций. В более сложных случаях, когда полного паросочетания из нулей нет, потребуется пересчёт.

5. Анализ сложности

Классическая реализация венгерского алгоритма имеет трудоёмкость $O(n^3)$. Каждая итерация (построение дерева и возможная модификация) может занимать $O(n^2)$, а число модификаций не превышает n (так как на каждом шаге хотя бы один новый ноль добавляется на границе). Поиск дополняющего пути для каждой строки в худшем случае требует $O(n^2)$ операций. Итоговая сложность – $O(n^3)$.

Существуют оптимизированные версии, использующие приоритетные очереди (кучи) для ускорения поиска дельты и обновления потенциалов. Они достигают $O(n^2 \cdot \log n)$ или $O(n^3)$ с меньшей константой. Однако для плотных матриц $O(n^3)$ является приемлемым при n до нескольких тысяч.

6. Оптимизации и структуры данных

Можно возможность ускорить поиск минимального ребра ($delta$) с помощью куч. Идея: для каждого столбца j, не входящего в T, поддерживать минимальное значение $C[i][j]$ среди $i \in S$. Тогда $delta$ – это минимум по этим минимумам. При добавлении новых вершин в S значения могут уменьшаться, а при глобальном пересчёте потенциалов – синхронно изменяться. Можно реализовать кучу с операцией «уменьшить все ключи на константу» через глобальный сдвиг (lazy propagation). Такая реализация даёт сложность $O(n^2 \cdot \log n)$. Детали реализации выходят за рамки данного конспекта, но важно понимать, что венгерский алгоритм допускает эффективные реализации.

7. Вариации и обобщения

- **Максимизация** вместо минимизации: достаточно заменить $C[i][j]$ на $-C[i][j]$ (или вычитать из максимального элемента) и решать задачу минимизации.
- **Неквадратная матрица**: если работников больше, чем работ ($n > m$), можно добавить фиктивные работы с нулевой стоимостью (или очень большой стоимостью для запрета), либо наоборот. Алгоритм легко адаптируется.
- **Запрещённые назначения**: можно установить очень большую стоимость (INF) для запрещённых пар.
- **Венгерский алгоритм для задачи о максимальном паросочетании минимального веса** в двудольном графе с полным взвешиванием.

8. Сравнение с другими методами

Метод	**Сложность**	**Применимость**
Полный перебор	$O(n! \cdot n)$	$n \leq 10$
Сведение к потоку минимальной стоимости	$O(n^3)$ или $O(flow \cdot n)$	универсально, но сложнее
Венгерский алгоритм	$O(n^3)$ / $O(n^2 \cdot \log n)$	оптимально для плотных матриц

Венгерский алгоритм остаётся одним из самых популярных методов для задачи о назначениях благодаря простоте реализации (в классическом варианте) и хорошей производительности на практике.